

C++ Design Document

GROUP C11 · Kolkata · Visteon Hackathon

Aaditya Kumar Choudhary · Jitendra Singh · Sneha Lahiri
Snigdhendu Chattopadhyay · Tamanna Parween

1. Architecture Explanation

The Smart Cabin & Vehicle Health Monitoring System is organised as a **pipeline of four concurrent stages**, each executing on its own OS thread and communicating through a mutex-protected shared data hub. The application is implemented entirely in C++17 with no external dependencies, following Adaptive AUTOSAR-inspired service-oriented principles.

1.1 High-Level Component Map

The diagram below shows the six logical layers and how data flows between them:

Layer	Class / File	Automotive Equivalent	Responsibility
Sensor Framework	Sensor.hpp / .cpp	MCAL / Sensor Abstraction Layer	6 virtual sensors; polymorphic update() every 500 ms
Alert Manager	Alert.hpp / .cpp	Vehicle Health Manager (VHM)	Threshold evaluation, severity assignment, deduplication
Event Logger	Logger.hpp / .cpp	Diagnostic Event Manager (DEM)	Async producer-consumer log pipe to vehicle_log.txt
Dashboard	Dashboard.hpp / .cpp	Instrument Cluster / HMI	ANSI-coloured ASCII render every 2 s
Thread Manager	ThreadManager.hpp / .cpp	Adaptive AUTOSAR Exec Context	Spawns / joins 4 worker threads; owns SharedVehicleData
JSON Parser	JsonParser.hpp	—	Header-only recursive-descent parser for test_cases.json

1.2 Thread Pipeline Architecture

main() constructs every module on the stack, moves the sensor vector zero-copy into **SharedVehicleData**, then hands control to **ThreadManager** which spawns four background workers:

Thread	Period	Work Performed	Mutex Held For
sensorThread	500 ms	update() all 6 sensors; push stats summary to logQueue	~<1 ms per cycle
monitorThread	1 000 ms	evaluateSensors(); clearExpiredAlerts(); push alert strings	~<1 ms per cycle
dashboardThread	2 000 ms	Render full ANSI table to stdout	~<5 ms per render
loggerThread	1 000 ms	Drain SafeQueue → logger.processQueue() → flush to disk	Never acquires dataMutex

1.3 Sensor Class Hierarchy

All sensors derive from the **abstract base class Sensor** that exposes three pure-virtual methods. Runtime polymorphism via vtable dispatch allows new sensor types to be added without modifying any consumer code

(Open/Closed Principle).

Sensor	Unit	Normal Range	Alert Threshold	Severity
EngineTemperatureSensor	°C	50–130	> 110 °C	CRITICAL
BatteryVoltageSensor	V	8–14.5	< 10.0 V	HIGH
SpeedSensor	km/h	0–160	> 120 km/h	HIGH
TirePressureSensor	PSI	20–40	< 25 PSI	LOW
DoorSensor	—	CLOSED/OPEN	OPEN + speed > 10 km/h	CRITICAL
SeatbeltSensor	—	LOCKED/UNLOCKED	UNLOCKED + moving	MEDIUM

1.4 Key Design Decisions

- **Abstract Base Class for Sensors**

Runtime polymorphism via virtual dispatch allows new sensor types by inheritance alone; no consumer-code changes required. vtable overhead (~8 B/object) is negligible for 6 sensors at 2 Hz.

- **Single dataMutex Guarding All Shared State**

With only 4 threads and critical sections under 1 ms, one mutex eliminates deadlock risk from lock-ordering bugs while still providing correct protection.

- **SafeQueue<T> for Log Decoupling**

Disk I/O latency (1–100 ms) is unpredictable. Pushing strings into a lock-free queue and draining on a separate thread guarantees sensor updates never stall waiting for fwrite().

- **std::atomic<bool> for Shutdown**

The running flag is read every loop iteration by 4 threads but written only once. An atomic load is essentially free; a mutex would add unnecessary contention on the hot path.

- **Rule-of-Five for Alert Value Type**

Alerts live in STL deque/vector and need correct copy/move. Explicit Rule of Five ensures the static alertCount_ tracker stays accurate across all container operations.

- **Header-Only EventLogger<T> Template**

Full implementation in Logger.hpp with explicit instantiation in Logger.cpp for std::string — avoids linker ODR violations while enabling generic reuse with any streamable type.

2. Thread Synchronization Explanation

The system uses three complementary synchronization mechanisms to ensure correctness, avoid deadlocks, and maintain low-latency operation:

1. **std::mutex + std::lock_guard** — protects all shared mutable state in SharedVehicleData (sensors, alertManager, statistics, dashboard).
2. **std::atomic<bool> running** — lock-free shutdown signalling polled at the top of every thread's main loop.
3. **SafeQueue<T>** — independent thread-safe FIFO with its own mutex and condition_variable, decoupling log producers from the logger consumer.

2.1 dataMutex — Shared State Protection

Every thread that reads or writes sensors, alertManager, or statistics must acquire **dataMutex** first. The RAIL pattern is used consistently — the lock is always released *before* the thread sleeps, minimising contention:

```
{ std::lock_guard<std::mutex> lock(data->dataMutex); /* critical section */ } // released here automatically – sleep happens OUTSIDE the lock
```

2.2 atomic<bool> running — Lock-Free Shutdown

The **running** flag uses relaxed atomic reads — essentially free on modern CPUs. On stop(), ThreadManager stores false and joins each thread:

```
data->running.store(false); // one write at shutdown for (auto& t : threads_) if (t.joinable()) t.join(); // wait for clean exit
```

2.3 SafeQueue<T> — Producer-Consumer Log Buffer

The SafeQueue exposes three access patterns, eliminating any coupling between disk I/O and the sensor/monitor hot path:

Method	Behaviour	Used By
push(T value)	Enqueue + notify one waiter via condition_variable	sensorThread, monitorThread
tryPop(T& value)	Non-blocking dequeue; returns false if empty	loggerThread (polling)
waitPop(T& value, timeout)	Blocking dequeue with predicate; handles spurious wakeups	Available; not on production path

2.4 Per-Thread Critical Section Analysis

■ sensorThread (500 ms)

Acquires dataMutex → calls sensor->update() for all 6 sensors → updates statistics → releases lock → optionally pushes status to logQueue (SafeQueue's own lock) → sleeps 500 ms.

■ monitorThread (1 000 ms)

Acquires dataMutex → evaluateSensors() → clearExpiredAlerts() → releases lock → pushes new alert strings to logQueue → sleeps 1 000 ms.

■ dashboardThread (2 000 ms)

Acquires dataMutex → renders full ANSI table to stdout (holds lock during render) → releases lock → sleeps 2 000 ms. Candidate for std::shared_mutex upgrade.

■ loggerThread (1 000 ms)

Never acquires dataMutex. Drains logQueue via tryPop() (SafeQueue's own lock) → calls logger.enqueue() (logger's own lock) → calls processQueue() to flush to disk → sleeps.

2.5 Shutdown Sequence

1. User presses ENTER in main()
2. main() calls threadManager.stop()
3. stop() sets running.store(false) ■ atomic, lock-free
4. Each worker thread exits its while(running.load()) loop at the next iteration
5. loggerThread performs final drain: pops all remaining messages, calls processQueue()
6. stop() calls t.join() for each thread — blocks until all finish
7. ThreadManager destructor clears threads_ vector
8. EventLogger destructor flushes remaining queue, writes footer, closes file
9. Sensor destructors decrement static sensorCount_
10. All resources released in reverse construction order (RAII stack unwinding)

3. Advanced / Bonus Features

The table below audits every advanced feature that could earn bonus marks, indicating whether it is fully implemented, partially implemented, or absent in the current codebase.

Feature	Status	Where / How Implemented
Multi-threaded sensor simulation (4 threads)	✓ YES	sensorThread, monitorThread, dashboardThread, loggerThread — all via ThreadManager
Asynchronous logging (producer-consumer)	✓ YES	SafeQueue<string> in ThreadManager.hpp; loggerThread drains to EventLogger<string>
RAII everywhere (no raw new/delete)	✓ YES	unique_ptr<Sensor> for sensors; lock_guard for mutexes; EventLogger closes file in dtor
Runtime polymorphism / virtual dispatch	✓ YES	Sensor abstract base with pure-virtual update(), getValue(), display() — 6 derived classes
Smart pointers (unique_ptr / shared_ptr)	✓ YES	unique_ptr<Sensor> in sensor vector; shared_ptr<SharedVehicleData> across all threads
Class Templates (generic reuse)	✓ YES	EventLogger<T> (header-only) and SafeQueue<T> — explicit std::string instantiation in Logger.cpp
STL containers (vector / deque / map / set)	✓ YES	vector<thread>, deque<Alert> (active), vector<Alert> (history), map<string, SensorStats>
Lambda expressions	✓ YES	std::copy_if predicate in search(); std::find_if in evaluateSensors(); condition_variable predicate
Move semantics (Rule of Five)	✓ YES	Alert class: explicit copy-ctor, move-ctor, copy-assign, move-assign, dtor; sensor vector moved into SharedVehicleData
Operator overloading	✓ YES	operator<< on Alert, Sensor, VehicleStatistics; operator< and == on Alert for STL algorithms
Exception handling for file / bad data	✓ YES	EventLogger ctor/dtor wrapped in try-catch; JsonParser throws on malformed input; Alert validates thresholds
Atomic shutdown flag	✓ YES	std::atomic<bool> running in SharedVehicleData — lock-free polling in all 4 thread loops
condition_variable for efficient blocking	✓ YES	SafeQueue::waitPop() uses cv_.wait_for() with predicate — no busy-wait
std::filesystem for portable I/O	✓ YES	main.cpp uses std::filesystem::create_directories() to ensure logs/ exists before first write
Structured bindings (C++17)	✓ YES	for (const auto& [name, s] : allStats) in Dashboard::display() for map iteration
constexpr thresholds	✓ YES	AlertManager: constexpr float ENGINE_TEMP_CRITICAL = 110.0f; etc. — compile-time constants
Data-driven automated test suite (10 cases)	✓ YES	tests.cpp: JSON-loaded scenarios, CSV reporting, __rdtsc cycle profiling — PerformanceProfiler class

Feature	Status	Where / How Implemented
Custom header-only JSON parser	✓ YES	JsonParser.hpp: self-contained recursive-descent parser; zero external dependencies
Per-sensor independent RNG (std::mt19937)	✓ YES	Each sensor owns private mt19937 seeded via random_device + atomic counter — no correlated readings
5% fault injection (spike / dip / deflation)	✓ YES	Gaussian base ± Bernoulli fault model in each sensor's update(); configurable via config.txt
Graceful shutdown on SIGINT / ENTER	✓ YES	Signal handler sets running=false; ThreadManager::stop() joins all threads before exit
Smart log viewer CLI tool (slog)	✓ YES	Bash slog script: print-all / tail -f / filter by severity / search by phrase / --clear
Binary size profiling (code_size_metrics.txt)	✓ YES	Per-module .text/.data/.bss segments documented; total linked binary ~1.51 MB
std::shared_mutex for dashboard reads	✗ NO	Not implemented. dashboardThread holds exclusive lock during renders. Noted as improvement in design doc.
Coroutines / co_await for sensor updates	✗ NO	Not implemented — C++20 feature; project targets C++17
Network socket output / MQTT	✗ NO	Not implemented — out of scope for hackathon console application

Summary: 24 of 27 audited features are fully implemented. 3 features (shared_mutex, coroutines, networking) are not applicable or intentionally deferred as C++20/out-of-scope improvements.

4. Mandatory C++ Concepts — Implementation Details

The table below maps every mandatory C++ concept listed in the hackathon rubric to its concrete implementation location and demonstrates compliance.

C++ Concept	Mandatory Usage	Where Implemented	Detail
Classes & Objects	Sensor, Alert, Dashboard	Sensor.hpp, Alert.hpp, Dashboard.hpp	Encapsulated classes with private data, public interfaces, and constructors managing invariants
Constructors / Destructors	Resource management	All major classes	EventLogger ctor opens file; dtor flushes & closes. Sensor ctor seeds RNG and increments sensorCount_
RAII	File handling / ownership	EventLogger, lock_guard, ThreadManager	std::lock_guard releases mutex on scope exit. EventLogger guarantees file close even on exceptions
Inheritance	Base Sensor class	Sensor.hpp → 6 derived classes	EngineTemperatureSensor, BatteryVoltageSensor, SpeedSensor, TirePressureSensor, DoorSensor, SeatbeltSensor all extend Sensor
Virtual Functions	Runtime polymorphism	Sensor::update() = 0, getValue() = 0, display() = 0	vtable dispatch in sensorThread: for(auto& s : sensors) s->update() calls the correct derived implementation
STL Containers	vector, map, set	ThreadManager.hpp, Alert.hpp, Dashboard.hpp	vector<unique_ptr<Sensor>>, deque<Alert> (active), vector<Alert> (history), map<string, SensorStats> (statistics)
Templates	Generic utility / logger	Logger.hpp, ThreadManager.hpp	EventLogger<T> header-only template; SafeQueue<T> lock-based producer-consumer queue — both reusable with any type
Smart Pointers	unique_ptr / shared_ptr	main.cpp, ThreadManager.hpp	unique_ptr<Sensor> gives exclusive sensor ownership; shared_ptr<SharedVehicleData> keeps data alive across threads
Exception Handling	File failures / invalid data	Logger.cpp, JsonParser.hpp, Alert.cpp	try-catch around file open in EventLogger; JsonParser throws std::runtime_error on malformed JSON; Alert validates numeric thresholds

4.1 Additional C++17 Features Used

Beyond the mandatory eight concepts, the implementation leverages several modern C++17 features for correctness and readability:

Feature	Location	Purpose
std::atomic<bool>	SharedVehicleData::running	Lock-free shutdown flag; polled at zero cost in every thread loop

Feature	Location	Purpose
<code>std::condition_variable</code>	<code>SafeQueue::waitPop()</code>	Efficient blocking wait — no busy-loop; predicate guards against spurious wakeups
<code>std::filesystem</code>	<code>main.cpp</code> — log directory creation	Portable <code>std::filesystem::create_directories()</code> for logs/ folder
<code>std::chrono</code>	Alert timestamps, thread sleep	High-resolution timing; used for timestamped log entries and <code>sleep_for()</code> calls
Lambda expressions	<code>filterAlerts()</code> , <code>evaluateSensors()</code>	Inline predicates for <code>std::copy_if</code> (log search) and <code>std::find_if</code> (sensor lookup)
Structured bindings	<code>Dashboard::display()</code> map loop	<code>for (const auto& [name, s] : allStats)</code> for clean map iteration
<code>constexpr</code> constants	AlertManager thresholds	<code>ENGINE_TEMP_CRITICAL</code> , <code>BATTERY_LOW</code> , etc. — compile-time constant folding
Move semantics	Sensor vector → <code>SharedVehicleData</code>	<code>std::move()</code> transfers sensor vector without deep-copying 6 <code>unique_ptr</code> objects
Operator overloading	Alert, Sensor, <code>VehicleStatistics</code>	<code>operator<<</code> for stream output; <code>operator<</code> and <code>==</code> for STL algorithm compatibility
enum class	<code>AlertSeverity</code>	Scoped, type-safe severity enumeration: <code>INFO</code> → <code>LOW</code> → <code>MEDIUM</code> → <code>HIGH</code> → <code>WARNING</code> → <code>CRITICAL</code>
<code>#pragma once</code>	All headers	Portable include guard replacing traditional <code>#ifndef/#define/#endif</code> guards